

CHAPTER 16

16 - 상품 이미지 + 배송 상태머신

Node.js · Express · NestJS 강의 · 2026

쇼핑몰의 거래 흐름은 15장에서 완성했다. 이 장은 거기에 두 가지를 더한다.

- **상품 이미지** — 판매자가 상품에 이미지를 올리고 내려받는다. 로컬 디스크 저장(multer).
- **배송 상태머신** — 결제된 주문을 관리자가 PAID → SHIPPED → DONE 순서로만 넘긴다. 건너뛰기는 막는다.

이 장에서 달라진 것 (← 15장 대비)

구분	파일 / 내용
신규	<code>common/upload.config.ts</code> (multer 설정) · <code>orders/order-status.ts</code> (전이 규칙) · <code>orders/dto/update-status.dto.ts</code> · <code>uploads/</code> (저장 폴더)
수정	<code>prisma/schema.prisma</code> — ProductImage 모델 + Product.images · <code>products</code> — 이미지 업로드/삭제 + 응답에 <code>url</code> · <code>orders</code> — 배송 상태 변경(상태머신) · <code>app.module</code> 에 <code>ServeStaticModule</code> · <code>main.ts</code> 에서 업로드 폴더 생성
의존성	<code>+@nestjs/serve-static</code> · <code>+multer</code> · <code>+@types/multer</code>
엔드포인트	<code>+POST /products/:id/images</code> · <code>+DELETE /products/images/:imageId</code> · <code>+PATCH /orders/:id/status</code>

0. 사전 준비 — PostgreSQL + Redis

PostgreSQL과 Redis(14장에서 Homebrew로 설치)가 떠 있어야 한다.

```
brew services list          # redis 가 started 인지 확인 (아니면 brew services start redis)
```

1. 패키지 설치

```
npm i @nestjs/serve-static@5
npm i -D @types/multer
```

`FileInterceptor` 와 `multer` 본체는 `@nestjs/platform-express` (Nest 11)에 이미 들어 있다 — 타입 선언만 추가하면 된다.

2. 스키마 — ProductImage + 마이그레이션

배송 상태(SHIPPED / DONE)는 이미 15장 `OrderStatus` enum에 정의돼 있고, 이 장에서 처음 쓴다. 스키마에는 이미지 모델만 더한다.

```
model Product {
  // ... 기존 필드 그대로 ...
  images ProductImage[] // 16장에서 추가 (1:N)
}

// 업로드한 이미지의 메타데이터만 DB에. 실제 파일은 uploads/ 에 떨어진다.
model ProductImage {
  id Int @id @default(autoincrement())
  storedName String // 디스크에 저장된 무작위 파일명
  product Product @relation(fields: [productId], references: [id], onDelete: Cascade)
  productId Int
}
```

```
npx prisma migrate dev --name add-product-images
```

3. 코드 작성 순서 (새 모듈은 없다 — 기존 모듈에 더한다)

순 서	파일	내용
1	<code>src/common/upload.config.ts</code>	multer 설정 — 저장 위치 · 무작위 파일명 · 이미지 MIME 화이트리스트 · 5MB 제한
2	<code>src/products/products.controller.ts</code>	POST <code>/products/:id/images</code> (<code>FileInterceptor</code>) · DELETE <code>/products/images/:imageId</code>
3	<code>src/products/products.service.ts</code>	이미지 메타 저장/삭제 + 디스크 정리(고아 방지) + 캐시 무효화, 응답에 <code>url</code>
4	<code>src/orders/order-status.ts</code>	허용 전이 표 — PAID → SHIPPED → DONE
5	<code>src/orders/dto/update-status.dto.ts</code> · <code>orders.service.ts</code> · <code>orders.controller.ts</code>	PATCH <code>/orders/:id/status</code> — 전이 검증, ADMIN 전용
6	<code>src/app.module.ts</code> · <code>src/main.ts</code>	<code>ServeStaticModule</code> 로 <code>/uploads</code> 서빙 · 부팅 시 업로드 폴더 생성

각 파일의 코드와 설명은 해설서 16장에 있다.

4. 실행 + 동작 테스트 (curl)

```
npm run start:dev
```

업로드한 이미지는 uploads/ 에 저장되고 http://localhost:3000/uploads/<파일명> 으로 내려받는다.

```
BASE=http://localhost:3000
login() { curl -s -X POST $BASE/auth/login -H "Content-Type: application/json" \
  -d '{"email":"'${1}',"password":"'${2}'"' | jq -r .access_token; }
ADMIN=$(login admin@example.com admin123)

curl -s -X POST $BASE/auth/register -H "Content-Type: application/json" \
  -d '{"email":"s1@example.com","password":"secret123","name":"판매자1","role":"SELLER"}'
S1=$(login s1@example.com secret123)

# 상품 등록 후 이미지 업로드 (경로를 실제 이미지로). 이미지가 아니면 400.
curl -s -X POST $BASE/products -H "Authorization: Bearer $S1" -H "Content-Type:
application/json" \
  -d '{"name":"무선 이어폰","description":"x","price":89000,"stock":3,"categoryIds":[1]}'
curl -s -X POST $BASE/products/1/images -H "Authorization: Bearer $S1" -F
"image=@/path/to/image.png"

# 상세에 images(url)가 보인다 → 그 url로 내려받기
curl -s $BASE/products/1
# curl -O http://localhost:3000/uploads/<무작위이름>.png

# --- 배송 상태머신 ---
curl -s -X POST $BASE/auth/register -H "Content-Type: application/json" \
  -d '{"email":"b@example.com","password":"secret123","name":"구매자"}'
B=$(login b@example.com secret123)
curl -s -X POST $BASE/cart/items -H "Authorization: Bearer $B" -H "Content-Type:
application/json" -d '{"productId":1,"quantity":1}'
curl -s -X POST $BASE/orders/checkout -H "Authorization: Bearer $B"
curl -s -X POST $BASE/orders/1/pay -H "Authorization: Bearer $B" -H "Content-Type:
application/json" -d '{"method":"CARD"}'

# PAID에서 DONE 직행 → 409 (건너뛰기 금지). 순서대로면 통과.
curl -i -X PATCH $BASE/orders/1/status -H "Authorization: Bearer $ADMIN" -H "Content-Type:
application/json" -d '{"status":"DONE"}' # 409
curl -s -X PATCH $BASE/orders/1/status -H "Authorization: Bearer $ADMIN" -H "Content-Type:
application/json" -d '{"status":"SHIPPED"}'
curl -s -X PATCH $BASE/orders/1/status -H "Authorization: Bearer $ADMIN" -H "Content-Type:
application/json" -d '{"status":"DONE"}'
```

핵심 개념

파일 업로드 — 받은 파일을 그대로 믿지 않는다

- **파일명을 새로 짓는다**: 사용자가 보낸 이름은 충돌·경로 조작 위험이 있다. 무작위 이름 + 원래 확장자 (`upload.config.ts`).
- **형식·크기를 거른다**: 이미지 MIME 화이트리스트 + 5MB. 통과 못 하면 저장 자체를 막는다.
- **DB와 디스크를 함께 관리한다**: 메타는 `ProductImage`, 실제 바이트는 `uploads/`. 삭제 시 둘 다 지운다. 권한 거부 시 방금 저장된 파일을 되돌려 지운다(고아 방지).

상태머신 — 정해진 순서로만

PENDING →결제→ PAID →배송→ SHIPPED →완료→ DONE

허용 전이를 표로 두고(`order-status.ts`), 표에 없는 변경은 409로 막는다. 그래서 PAID 에서 곧장 DONE 으로 갈 수 없다. 배송 상태 변경은 관리자(ADMIN)만 한다 — 컨트롤러에서 `if (user.role !== "ADMIN")` 로 막는다(13장과 같은 방식).

핵심 파일

파일	역할
<code>common/upload.config.ts</code>	multer 저장 위치·파일명·형식·크기
<code>products/products.service.ts</code>	이미지 저장/삭제 + 디스크 정리 + 캐시 무효화
<code>orders/order-status.ts</code>	배송 전이 규칙
<code>orders/orders.service.ts</code>	<code>updateShippingStatus</code> — 전이 검증
<code>app.module.ts</code>	<code>ServeStaticModule</code> 로 <code>/uploads</code> 서빙

따라잡기 — 완성본으로 실행 (보조 경로)

수업의 기본 경로는 위처럼 직접 만드는 것이다. 따라오다 놓쳤다면 수업 저장소의 16/ 폴더(완성 코드)로 이 장의 끝 상태에서 다시 시작할 수 있다.

```
cd 16
npm install
cp .env.example .env          # 사용자·비밀번호를 본인 PostgreSQL 값으로 수정
npx prisma migrate dev --name init  # 이 폴더 전용 DB(nest_shop_16) - 자동 생성이 안 되면
DBBeaver로 먼저 만든다
npm run seed
npm run start:dev
```

다음 장

- **17 - 마무리(완성형):** 검색·미결제 주문 자동취소(@Cron)를 더해 쇼핑몰을 완성한다. 17장은 손으로 짜지 않고 완성본을 돌려보며 읽는 자료 제공 장이다.